# Apostila de Apoio — Aula 3

Autômatos Finitos (DFA e NFA) — reconhecendo padrões

Teoria da Computação

Março/2026

**Como usar**

Esta apostila foi escrita para quem está começando do zero. Você não precisa saber programar. Aqui, a palavra **string** significa apenas "um texto" (uma sequência de símbolos). Nos exemplos, vamos usar alfabetos pequenos como {0,1} e {a,b} para facilitar.
A ideia principal da Aula 3 é aprender a desenhar e "rodar" máquinas simples que aceitam ou rejeitam **strings**.

## Sumário


- Visão Geral
- Autômatos
- DFA
- NFA
- Exercícios graduais
- Erros comuns

## 1. Visão geral

Nesta aula, você vai aprender o que é um **autômato finito**: uma máquina com poucos estados que lê uma **string** símbolo por símbolo e decide se ela é válida (aceita) ou inválida (rejeita).
No final, você deve conseguir:

- Explicar o que é **estado** e por que ele é uma "memória pequena".
- Simular um **DFA** (determinístico) passo a passo em uma string.
- Entender a ideia de **NFA** (não determinístico) e como ele aceita (basta existir um caminho que aceite).
- Construir DFAs simples a partir de frases em português (ex.: "termina com 1", "tem pelo menos um 1").

**Regra de ouro**

Sempre pergunte: **quais strings este autômato deve aceitar?** Se você consegue listar exemplos aceitos e rejeitados, você está entendendo.

## 2. Autômato, DFA e NFA

**Ideia principal:** um autômato finito é uma máquina com poucos "modos" (estados). Ele lê a **string** um símbolo por vez. A cada símbolo, ele troca de estado. No final, se o estado onde parou é um estado final, a **string** é aceita.

### 2.1 O que é um autômato finito

Pense em um semáforo: ele tem poucos estados (verde, amarelo, vermelho). O semáforo muda de estado conforme o tempo. Um autômato muda de estado conforme os símbolos lidos.
O nome "**finito**" significa que o conjunto de estados é finito. Isso dá uma intuição importante: o autômato tem **memória limitada**. Ele não consegue, por exemplo, contar até um número arbitrariamente grande sem "estourar" estados.

**Vocabulário rápido**

**String** (cadeia): um texto feito de símbolos (ex.: 0101).
**Estado**: a "situação atual" da máquina.
**Aceitar**: terminar a leitura em um estado final.
**Rejeitar**: terminar a leitura em um estado que não é final (ou não conseguir continuar, no caso de NFA incompleto).

### 2.2 DFA (Determinístico): definição e interpretação

Um DFA (Deterministic Finite Automaton) é definido por cinco partes:

- **Q**: conjunto de estados (ex.: {q0, q1}).
- **Σ**: alfabeto (ex.: {0,1}).
- **δ:** função de transição — diz para onde ir ao ler um símbolo.
- **q0**: estado inicial.
- **F:** conjunto de estados finais (de aceitação).

O que torna o DFA "determinístico" é esta regra:

**Regra do DFA (sem pegadinha)**

Para **cada** estado e para **cada** símbolo do alfabeto, existe **exatamente uma** transição de saída.
Ou seja: a máquina nunca fica em dúvida sobre qual caminho seguir.

### 2.3 Como simular um DFA (passo a passo)

1. Comece no estado inicial q0.
2. Leia o primeiro símbolo da **string** e siga a transição **δ**(estado, símbolo).
3. Repita para cada símbolo, atualizando o estado.
4. Quando a **string** acabar, verifique: o estado atual está em **F**? Se sim, aceita; se não, rejeita.

## 2.4 Exemplo 1 (DFA) — Linguagem: “termina com 1”

Alfabeto Σ = {0,1}. Queremos aceitar todas as **strings binárias** cujo último símbolo é **1**.

**Ideia simples**: só precisamos lembrar qual foi o último símbolo lido. Então usamos dois estados:

- q0: “o último símbolo não foi 1” (ou ainda não li nada).
- q1: “o último símbolo foi 1”.

**Estados finais:** F = {q1}. (Se terminou em q1, então terminou com 1.)

| Estado | lê 0 | lê 1 |
|---|---|---|
| q0 | q0 | q1 |
| q1 | q0 | q1 |

**Como ler a tabela**

**Exemplo**: na linha q0, coluna “lê 1”, está q1. Isso significa: se estou em q0 e leio o símbolo 1, vou para q1.

**Simulação 1:** w = 1010

Começo em q0 → leio 1: vou para q1 → leio 0: vou para q0 → leio 1: vou para q1 → leio 0: vou para q0. Término em q0, que não é final → rejeita.

**Simulação 2:** w = 000101

q0 —0→ q0 —0→ q0 —0→ q0 —1→ q1 —0→ q0 —1→ q1. Término em q1 → aceita.

## 2.5 Exemplo 2 (DFA) — Linguagem: “número par de 1”

Agora queremos aceitar **strings binárias** com quantidade par de símbolos 1 (0,2,4,...).

**Ideia iniciante**: em qualquer momento, eu só preciso saber se já vi um número par ou ímpar de 1. Isso gera dois estados:

- **qPar**: vi quantidade par de 1 até agora (começo aqui).
- **qImpar**: vi quantidade ímpar de 1 até agora.

**Regra mental:** ler 0 não muda nada; ler 1 troca par ↔ ímpar.

| Estado | lê 0 | lê 1 |
|---|---|---|
| qPar | qPar | qImpar |
| qImpar | qImpar | qPar |

**Simulação 1**: w = 1100

Começo em qPar → leio 1: qImpar → leio 1: qPar → leio 0: qPar → leio 0: qPar. Término em qPar (final) → aceita.

**Simulação 2**: w = 1011

qPar —1→ qImpar —0→ qImpar —1→ qPar —1→ qImpar. Término em qImpar → rejeita.

### 2.6 NFA (Não determinístico): ideia sem mistério

Um **NFA** é um autômato que pode ter escolhas. Para um mesmo estado e símbolo, ele pode ter 0, 1 ou várias transições.

**Regra de aceitação do NFA**

Uma **string** é aceita se existir pelo menos um caminho de leitura que termina em um estado final. Se nenhum caminho chega a um final, a **string** é rejeitada.

Pense assim: o **NFA** "**tenta**" caminhos diferentes. Se algum dá certo, ele aceita.

### 2.7 Exemplo (NFA) — Linguagem: "contém 01"

Queremos aceitar **strings binárias** que tenham a **substring 01** em algum lugar.

**Estados (ideia simples):**

q0: ainda não encontrei "01".

q1: acabei de ver um 0 que pode iniciar "01".

q2: já encontrei "01" (estado final).

**Transições (descritas em português):**

De q0: lendo 1, contínuo em q0 (ainda não começou "01"). Lendo 0, eu posso continuar em q0 (ignorar este 0) ou ir para q1 (apostar que esse 0 é o começo de "01").

De q1: lendo 1, vou para q2 (encontrei "01"). Lendo 0, fico em q1 (este novo 0 também pode ser começo).

De q2: lendo 0 ou 1, fico em q2 (já encontrei, não "desencontro").

**Simulação intuitiva (sem "explodir" caminhos)**

**Teste w = 111000**: não aparece "01" em nenhum ponto → rejeita.

**Teste w = 11000111**: existe um ponto onde aparece 0 seguido de 1 → algum caminho chega em q2 → aceita.

**Observação**: mais adiante veremos que todo **NFA** pode virar um **DFA** equivalente (ele reconhece a mesma linguagem).

## 3. Exercícios graduais

Assuma Σ = {0,1}, a menos que o exercício diga o contrário.

**Nível 1 — Simular**

1) No DFA “termina com 1”, diga se aceita:
   a) 0 b) 1 c) 10 d) 1011.
2) No DFA “par de 1”, diga se aceita:
3) a) ε b) 1 c) 11 d) 1010.

**Nível 2 — Modelar**

4) Desenhe um DFA para “termina com 0”.
5) Desenhe um DFA para “contém pelo menos um 1”.

**Nível 3 — Um pouco mais**

6) Desenhe um DFA para “número par de 0”.
7) Desenhe um DFA para “começa com 1”.
8) Desafio: DFA para “termina com 00” (dica: você precisa lembrar os dois últimos símbolos).

## 5. Erros comuns

- Confundir estado final com “fim”: estado final é só um estado “bom”. Ele pode ter transições saindo dele.
- Esquecer transições no **DFA**: em **DFA** completo, para cada estado e cada símbolo, existe uma transição.
- Achar que “passar por final” aceita: no **DFA**, aceita se termina em final quando a **string** acaba.
- No **NFA**, achar que todos os caminhos precisam aceitar: basta um caminho chegar em estado final.
- Não fixar o alfabeto **Σ**: o desenho depende dos símbolos possíveis.

## 6. Glossário rápido

- Σ (alfabeto): conjunto finito de símbolos permitidos.
- símbolo: um elemento de Σ (por exemplo 0 ou 1).
- string/cadeia (w): sequência finita de símbolos de Σ.
- ε (épsilon): string vazia (comprimento 0).
- |w|: comprimento da string w.
- Σ*: conjunto de todas as strings finitas sobre Σ (inclui ε).
- L: linguagem (um conjunto de strings), L ⊆ Σ*.
- ER: expressão regular (notação para descrever linguagens regulares).
- L(r): linguagem descrita (denotada) pela ER r.
- |: operador “ou” (união).
- concatenação: colocar uma string após a outra (sequência).
- *: estrela de Kleene (repetição zero ou mais vezes).